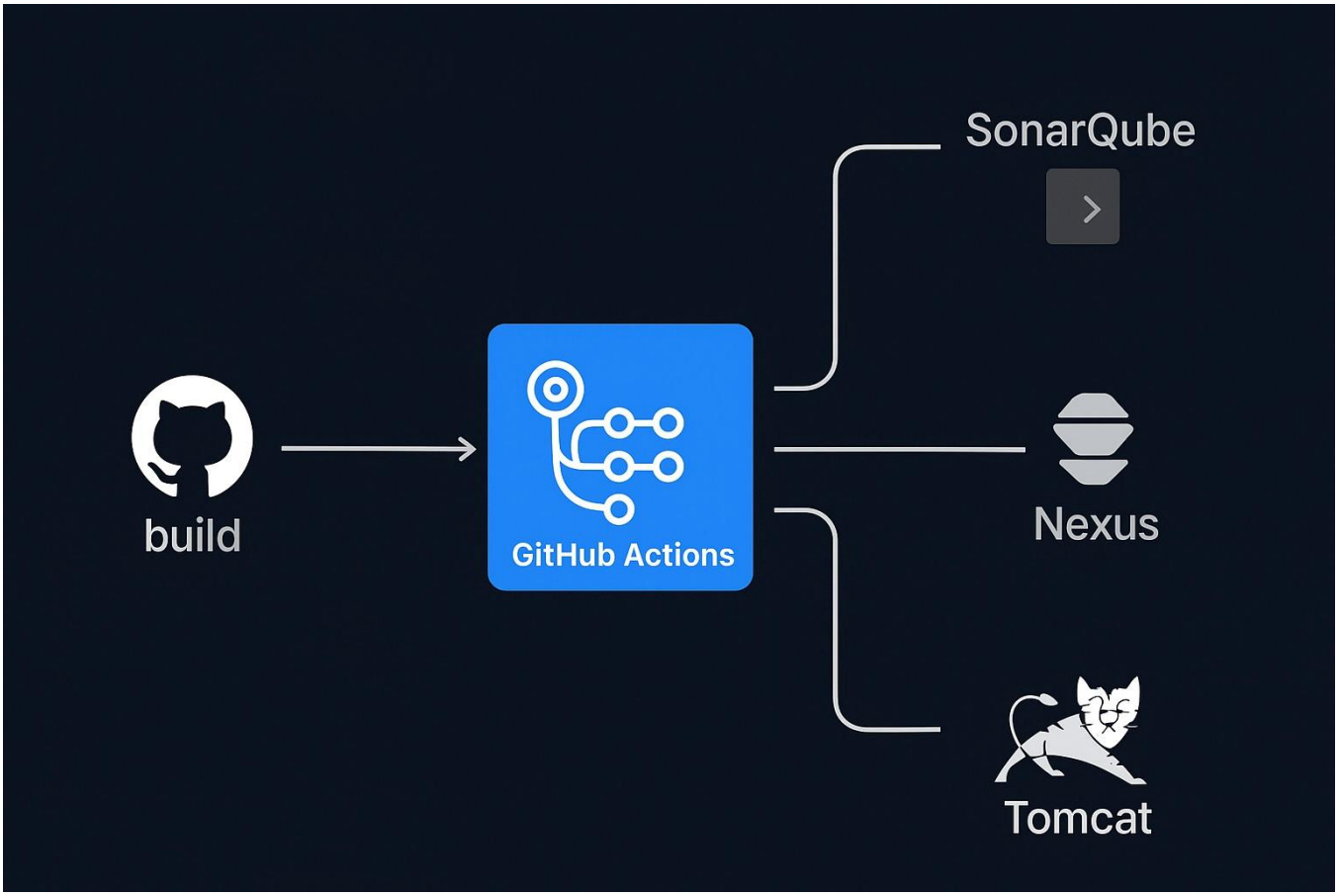
PROJECT – 5

Automated CI/CD Pipeline Using GitHub Actions with SonarQube, Nexus, and Tomcat

Presentation By : M.Karthik Reddy

Batch : 141

Automated CI/CD Pipeline Using GitHub Actions with SonarQube, Nexus, and Tomcat



Tools Used:

- **GitHub** - Source code manage
- **GitHub Actions** - CI/CD automation
- **Maven** - Build and dependency management
- **SonarQube** - Static code analysis & quality checks
- **Nexus Repository** - Artifact storage
- **Apache Tomcat** - Application deployment server
- **Java (JDK 17)** - Application runtime

INTRODUCTION

This project focuses on the **implementation of an automated CI/CD pipeline for a Java web application using GitHub Actions**. The pipeline integrates multiple DevOps tools to ensure consistent builds, code quality validation, artifact management, and automated deployment. The objective of this project is to design and implement a streamlined workflow that transforms source code changes into a deployable application with minimal manual intervention.

The CI/CD pipeline begins when a developer pushes code changes to a GitHub repository. GitHub Actions is used as the automation engine to orchestrate the pipeline stages. **Apache Maven** is utilized as the build and dependency management tool, executing various lifecycle phases such as validation, compilation, testing, packaging, and verification. This ensures that the application is built in a structured and repeatable manner.

To maintain code quality and enforce best practices, **SonarQube** is integrated into the pipeline. SonarQube performs static code analysis to identify bugs, code smells, and potential security vulnerabilities. By running quality checks early in the pipeline, the project ensures that only clean and maintainable code progresses further in the delivery process.

After successful build and analysis, the generated artifact (WAR file) is stored in **Nexus Repository**, which acts as a centralized artifact management system. Nexus provides version control for build artifacts, enabling consistency across different environments and preventing unnecessary rebuilds. This separation of build and deployment stages aligns with industry-standard DevOps practices.

Finally, the application is deployed to **Apache Tomcat**, a widely used Java application server. Deployment is automated using the Tomcat Manager Text API, allowing the pipeline to deploy or update the application without manual server access. This completes the continuous deployment phase and ensures faster and more reliable releases.

Overall, this project demonstrates a real-world CI/CD implementation that follows DevOps best practices by integrating source control, build automation, code quality analysis, artifact management, and automated deployment. The pipeline improves development efficiency, reduces human errors, and provides a scalable foundation that can be extended to support multi-environment deployments, security enhancements, and container-based architectures.

PROCESS OF DEPLOY WAR FILE IN TOMCAT USING GITHUB ACTIONS

STEP – 1: CREATE EC2 INSTANCES

- Create EC2 Instances for Sonarqube, Nexus, Tomcat or It can be install in 1 server.
- Tabel the names for different EC2 Instances (ex: Sonarqube-server, Nexus-server, Tomcat-server)

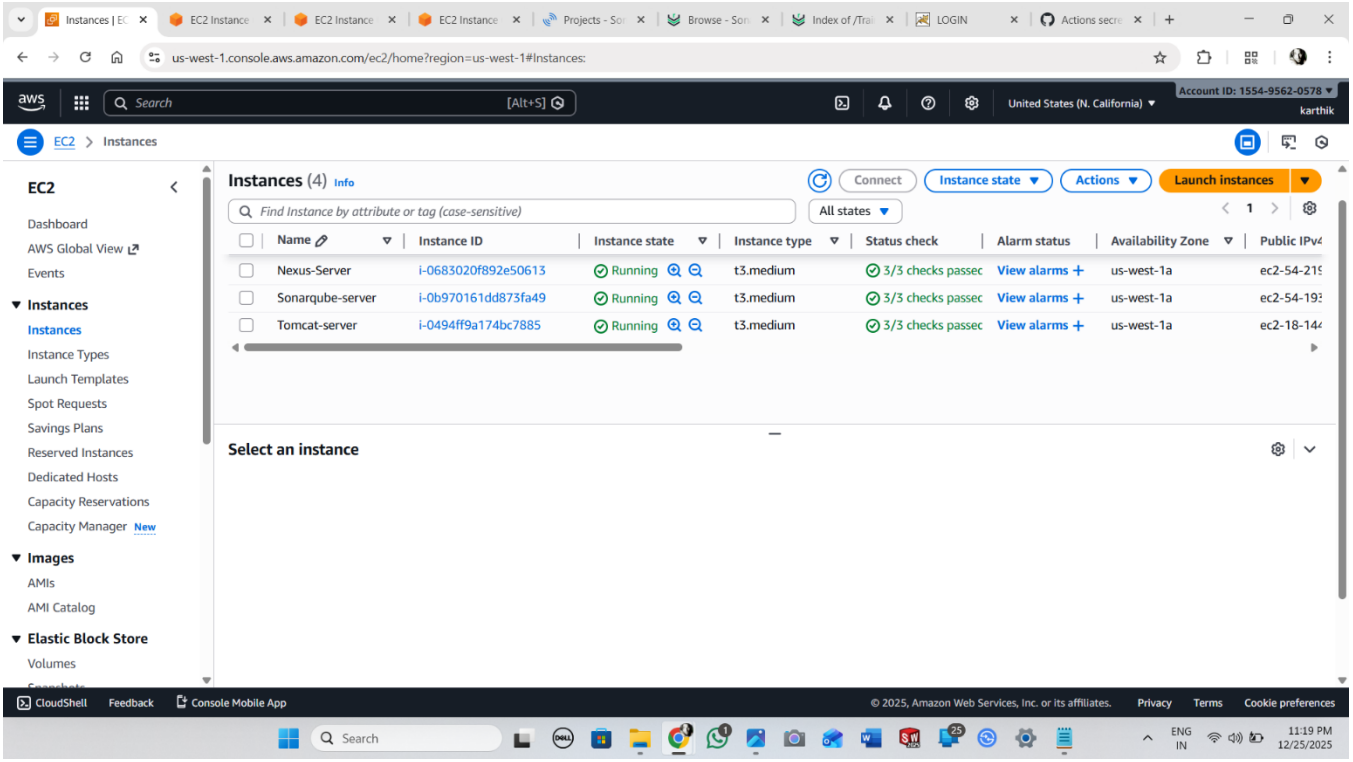


Fig-1: EC2 Instances

STEP – 2: INSTALL SONARQUBE ON EC2

- Run the command “apt update -y” before install the sonarqube on EC2 Instance
- Install sonarqube by using Docker
- Install Docker (“apt install docker.io -y”)
- Start the docker
- Create and run the container of sonarqube on EC2 Instance
  (“Docker run –run myc1 -d -p 9000:9000 sonarqube:latest”)

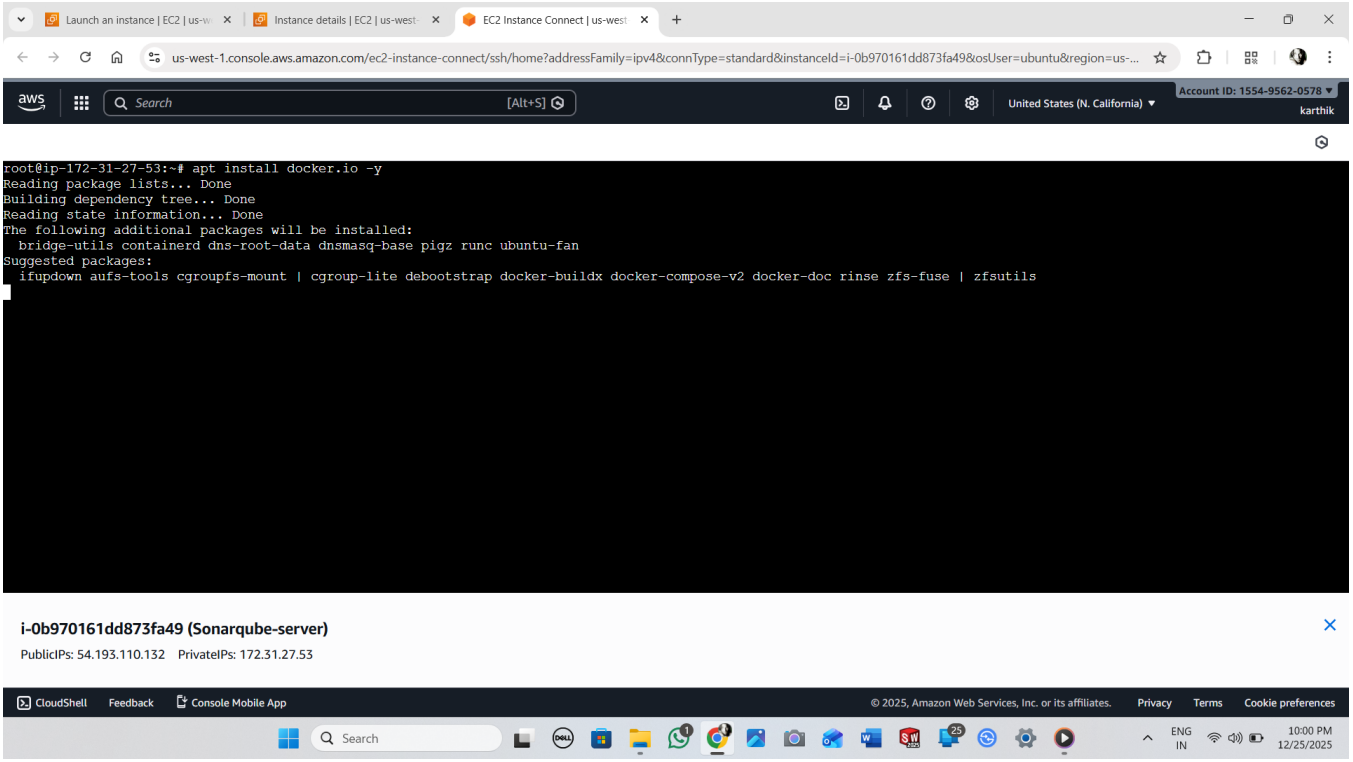


Fig-2: Docker

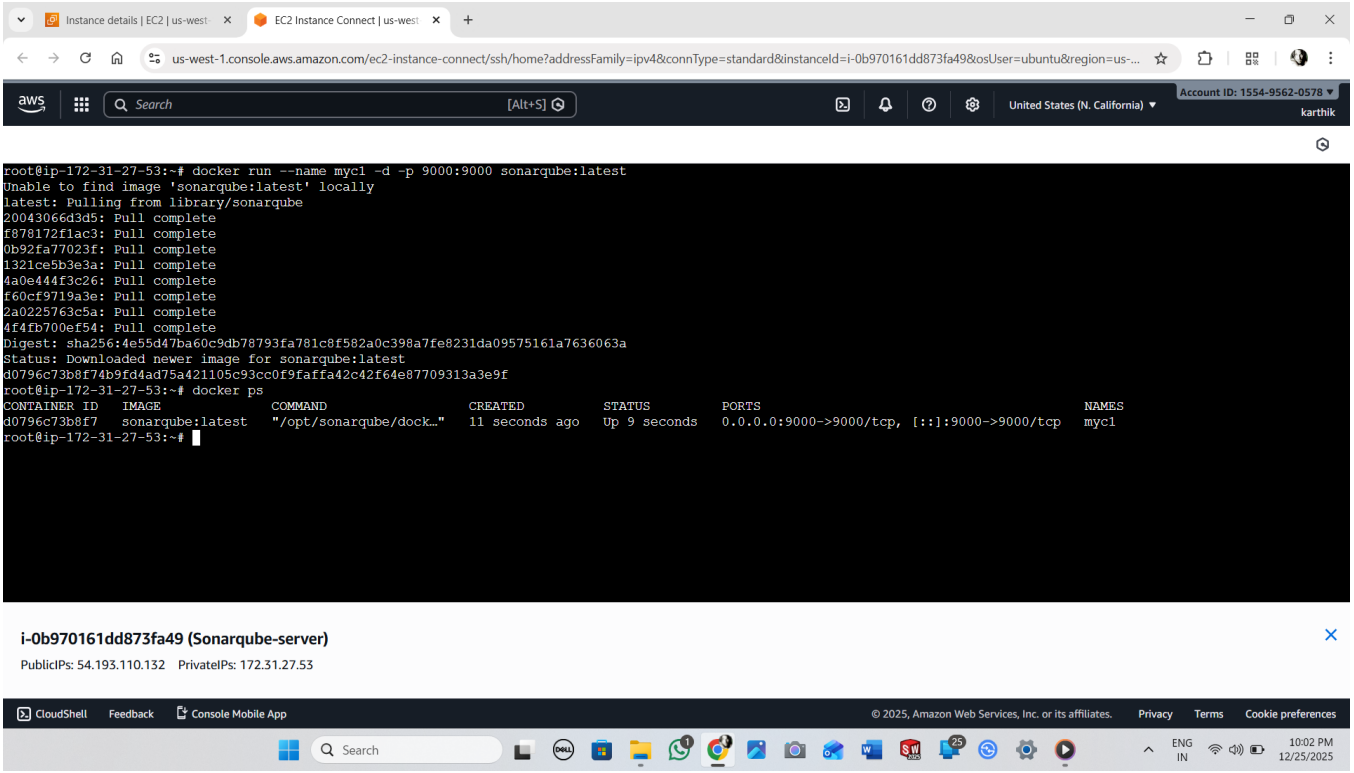


Fig-3: Install Sonarqube

STEP – 3: INSTALL NEXUS ON EC2

- Install Nexus by using Docker
- Create and run the container of Nexus on EC2 Instance

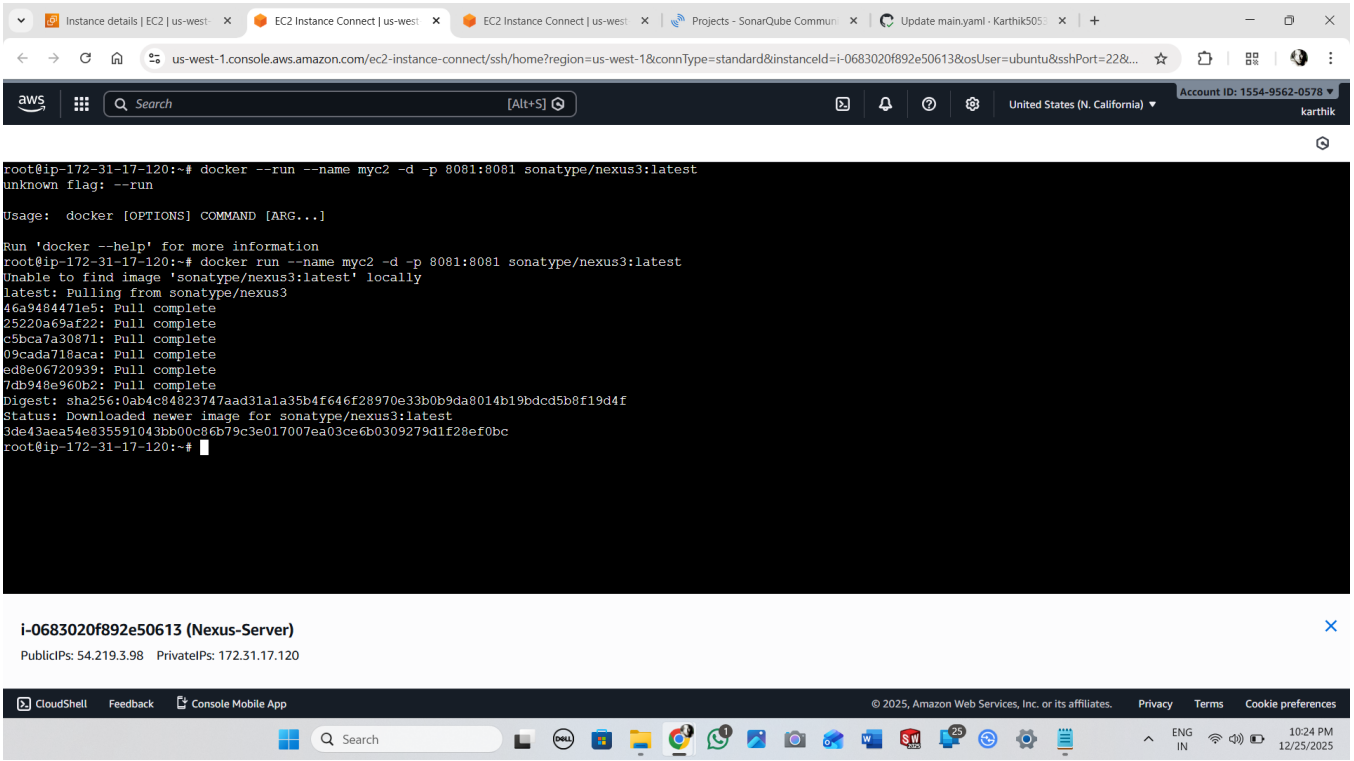


Fig-4: Install Nexus

STEP – 4: INSTALL TOMCAT ON EC2

- Get Tomcat9 On EC2 Instance

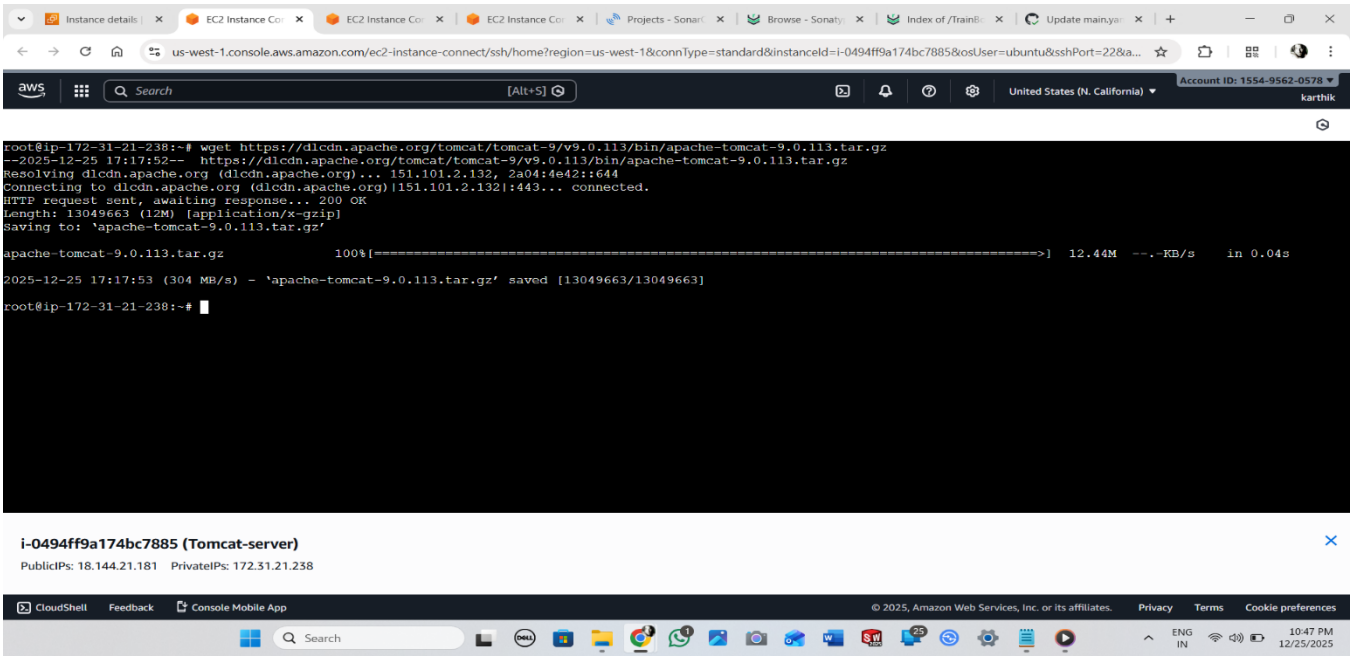


Fig-5: Get Tomcat

STEP-5: LOGIN & GENERATE TOKEN IN SONARQUBE

- Open the Sonarqube by using server public ip.
- Login to Sonarqube.
- Generate Token in sonarqube.

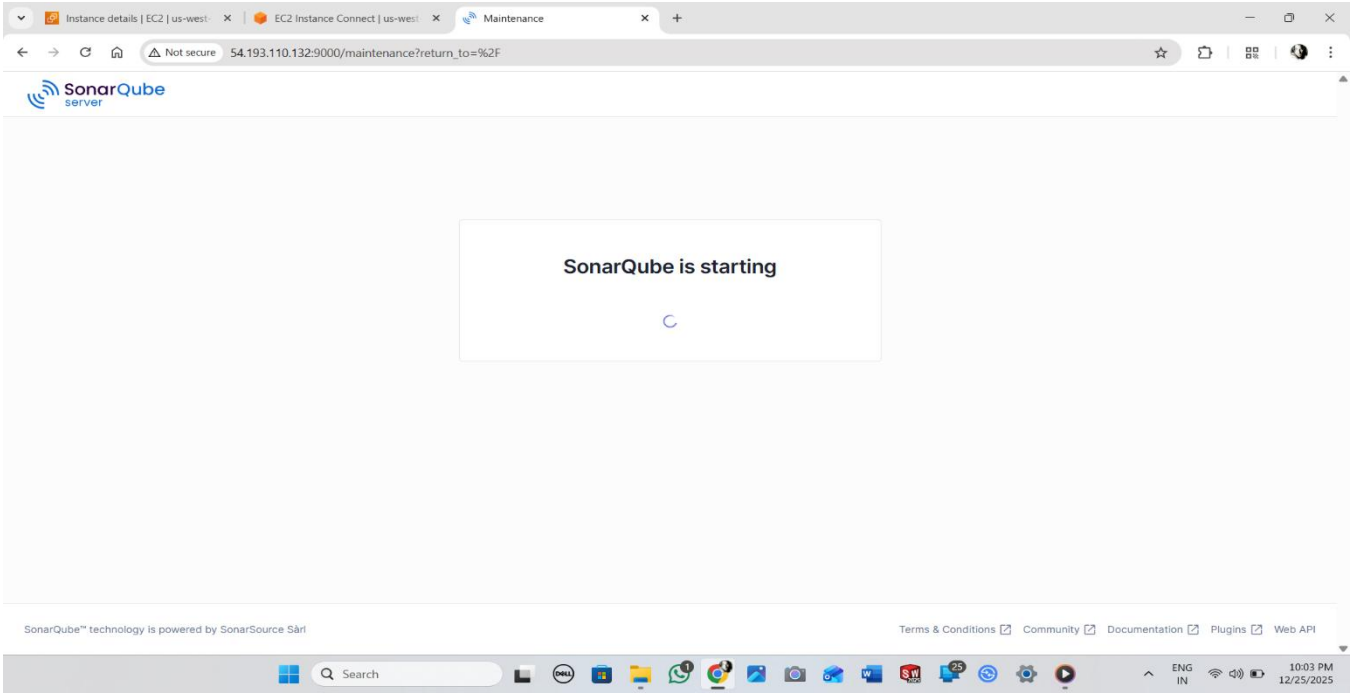


Fig-6: Open Sonarqube

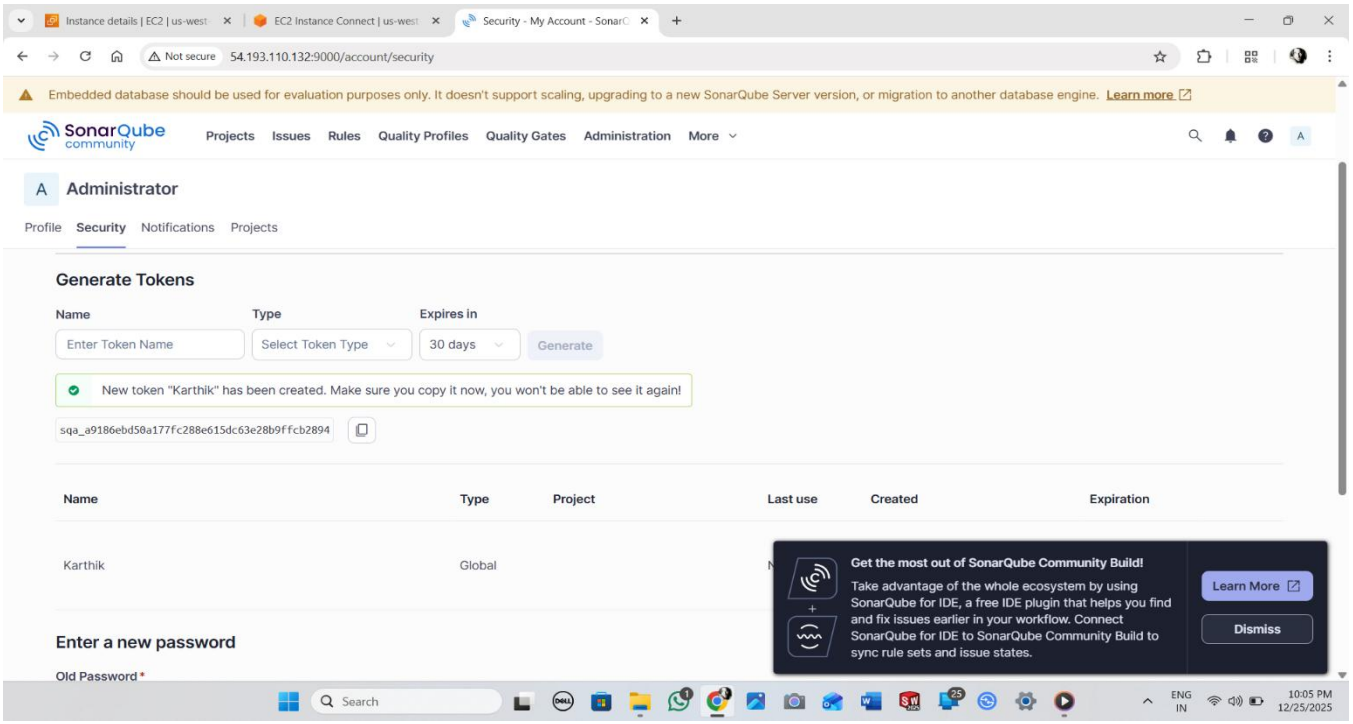


Fig-7: Generate Token

STEP-6: LOGIN TO NEXUS

- Open the Nexus by using server public ip.
- Login to Nexus
- Copy the Repo-url in nexus and note it.

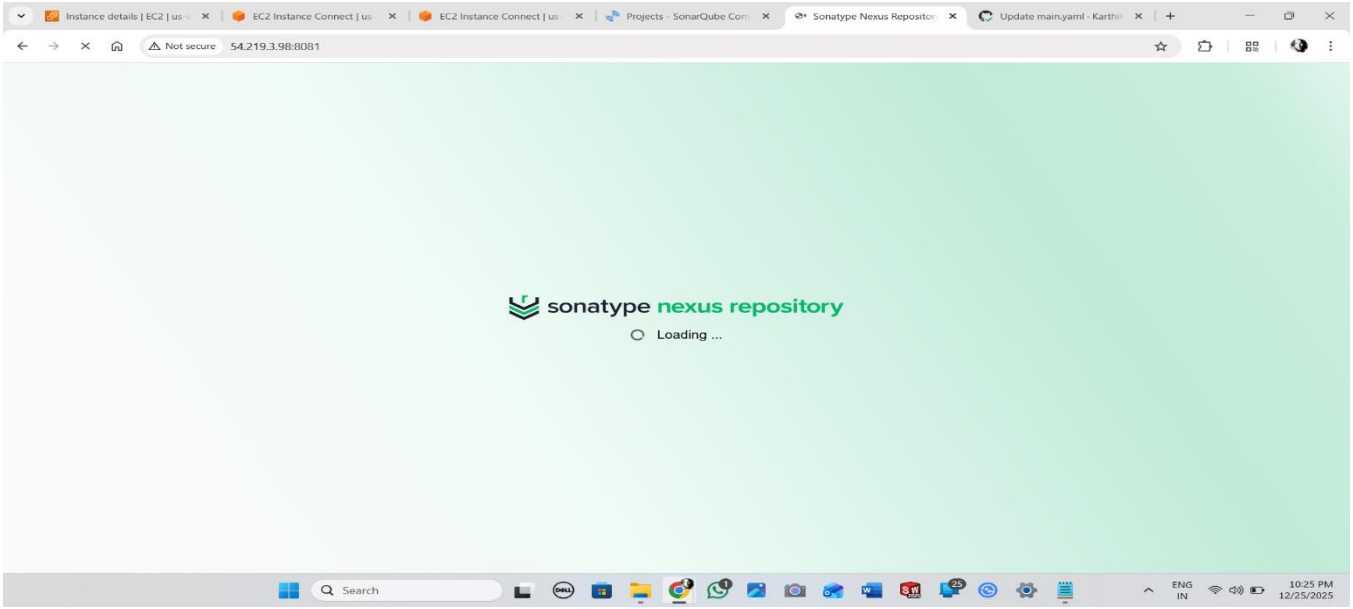


Fig-8: Nexus

STEP-7: LOGIN TO TOMCAT

- Install the tomcat 9 in all EC2 Instances
- Apache Tomcat was downloaded, extracted, and configured to run on port 8080.
- The tomcat-users.xml file was updated to create a user with manager-gui and manager-script roles.

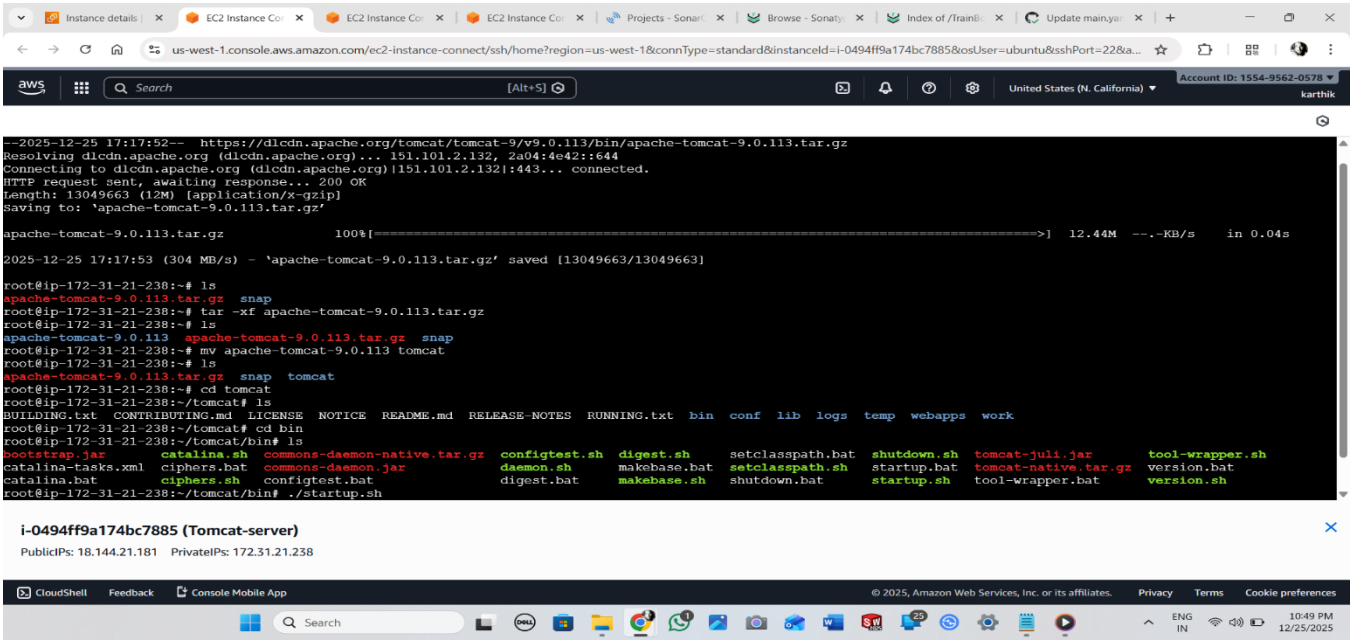


Fig-9: Tomcat 9 On EC2 Instances

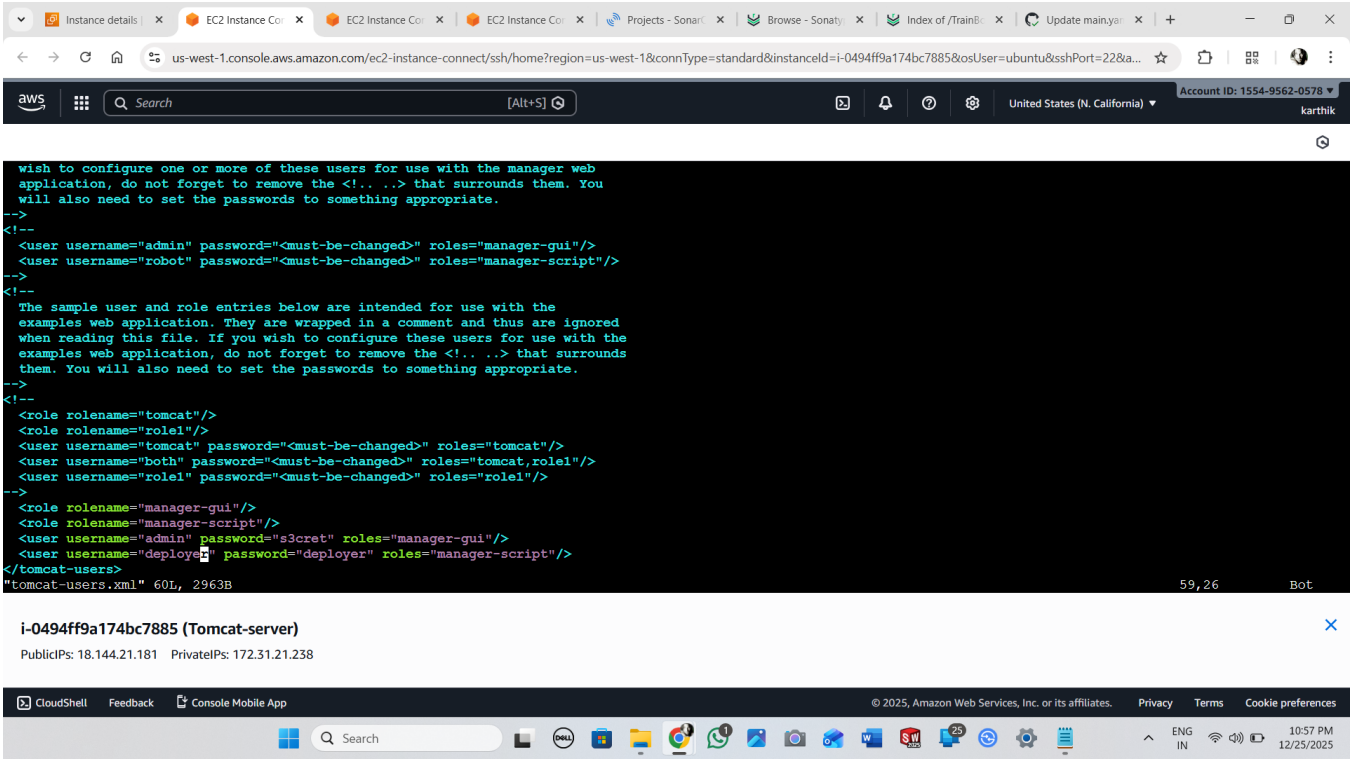


Fig-10: Tomcat-users.xml file updated with manager-gui and manager-script roles

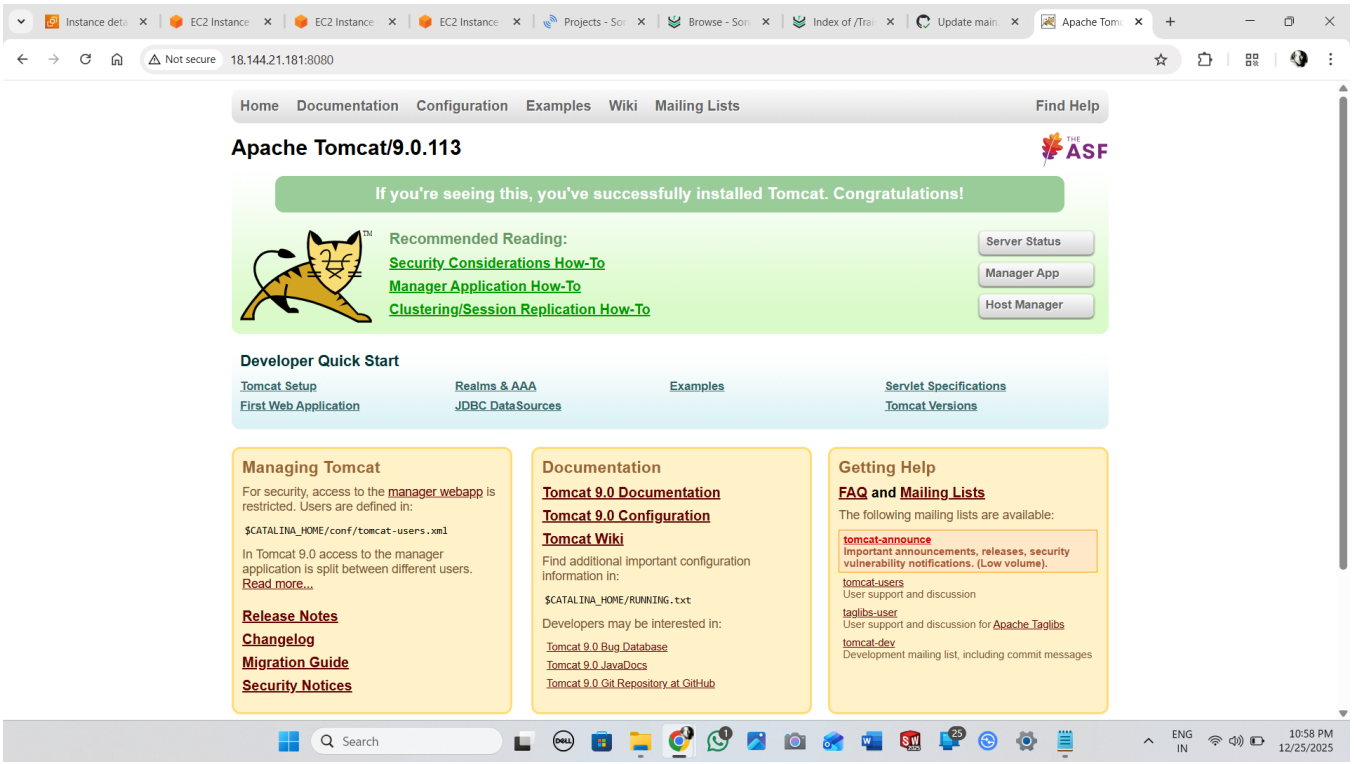


Fig-11: Tomcat 9

STEP – 8: CONFIGURE GITHUB REPOSITORY

- Source code in a GitHub repository.
- Create .github/workflows/main.yaml file was created to define the CI/CD pipeline.
- Add Secrets actions repository for CI/CD pipeline.

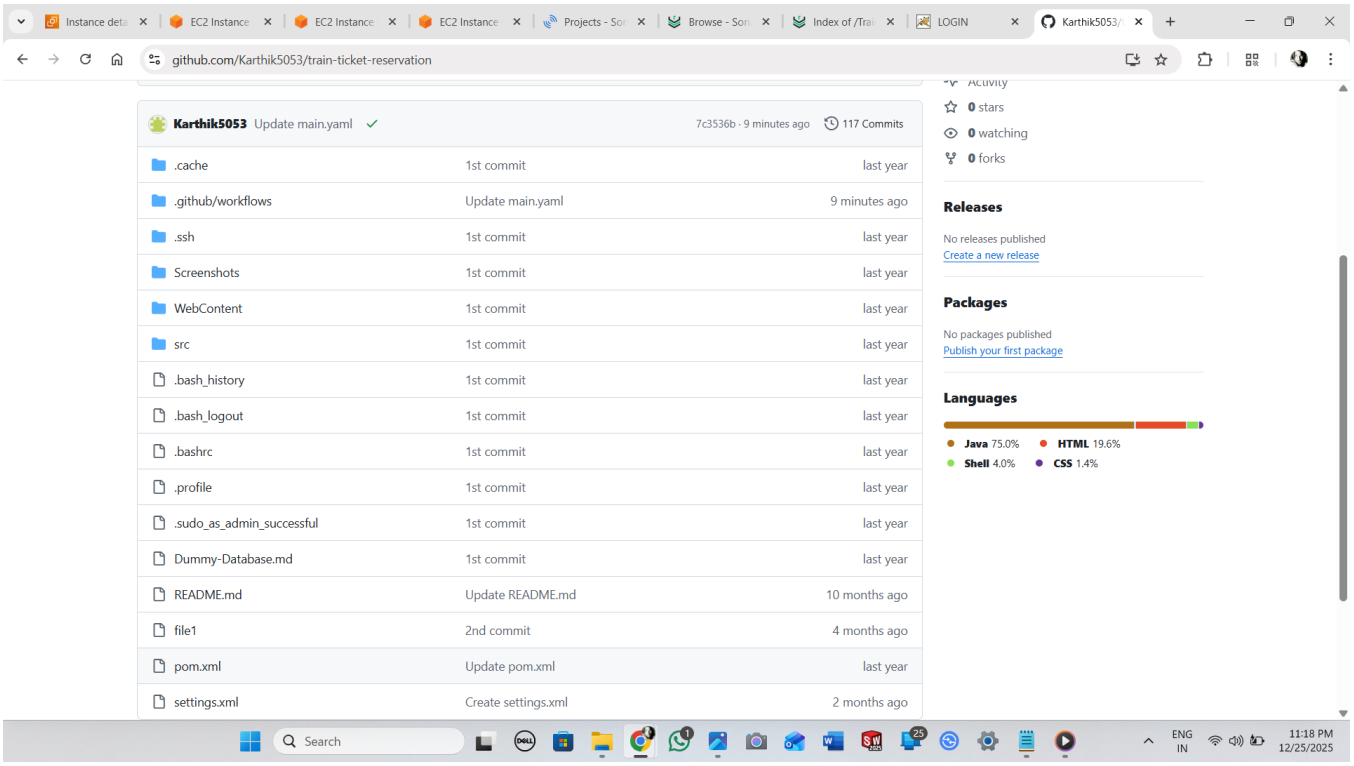


Fig-12: Source code in a GitHub repository

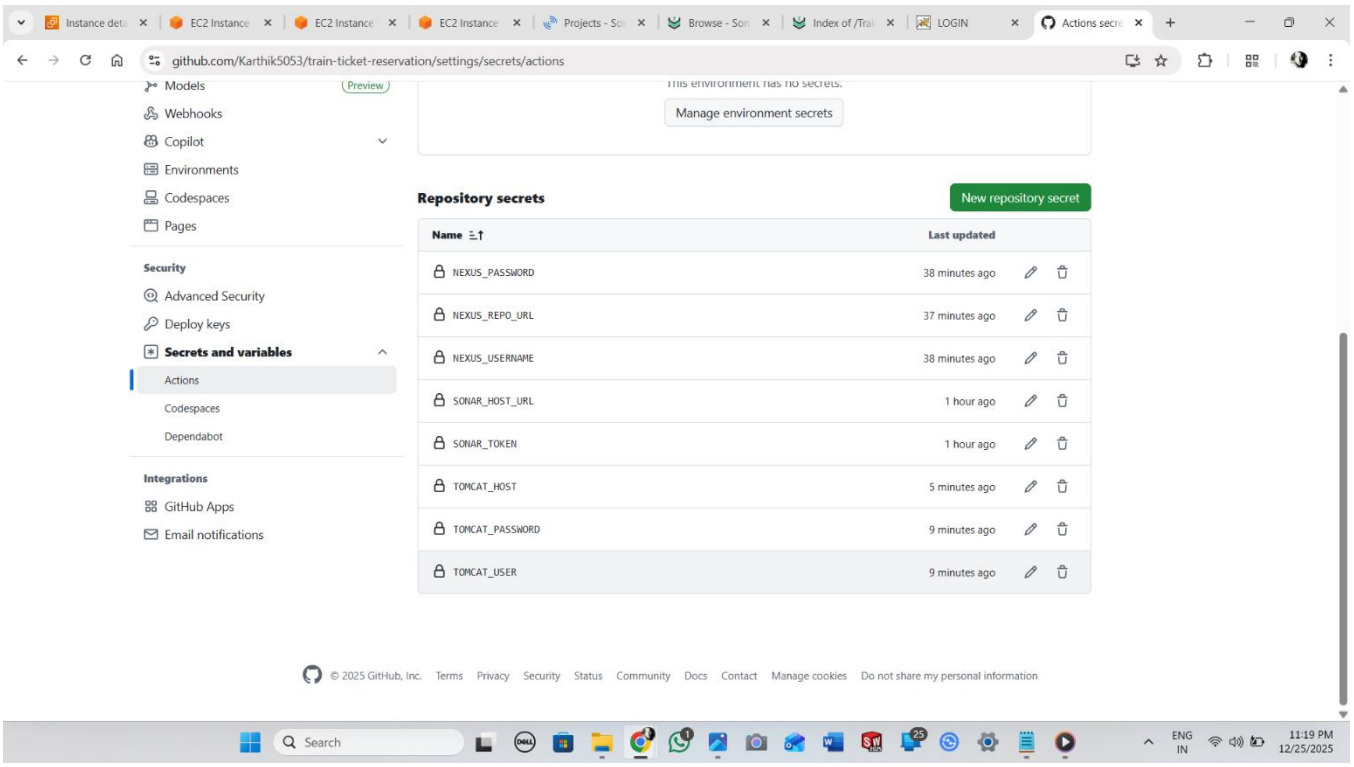


Fig-13: Secrets actions repository

STEP – 9: CI/CD PIPELINE USING GITHUB ACTIONS

- The pipeline checks out the code.
- Sets up Java and runs Maven commands (validate, compile, test, package).
- Uploads the WAR file as an artifact.
- Downloads the WAR in deploy jobs and uses curl to deploy to Tomcat using the Manager API.
- GitHub Secrets store different hostnames and credentials.
- Deploy jobs run in sequence: Sonarqube → Nexus → Tomcat.

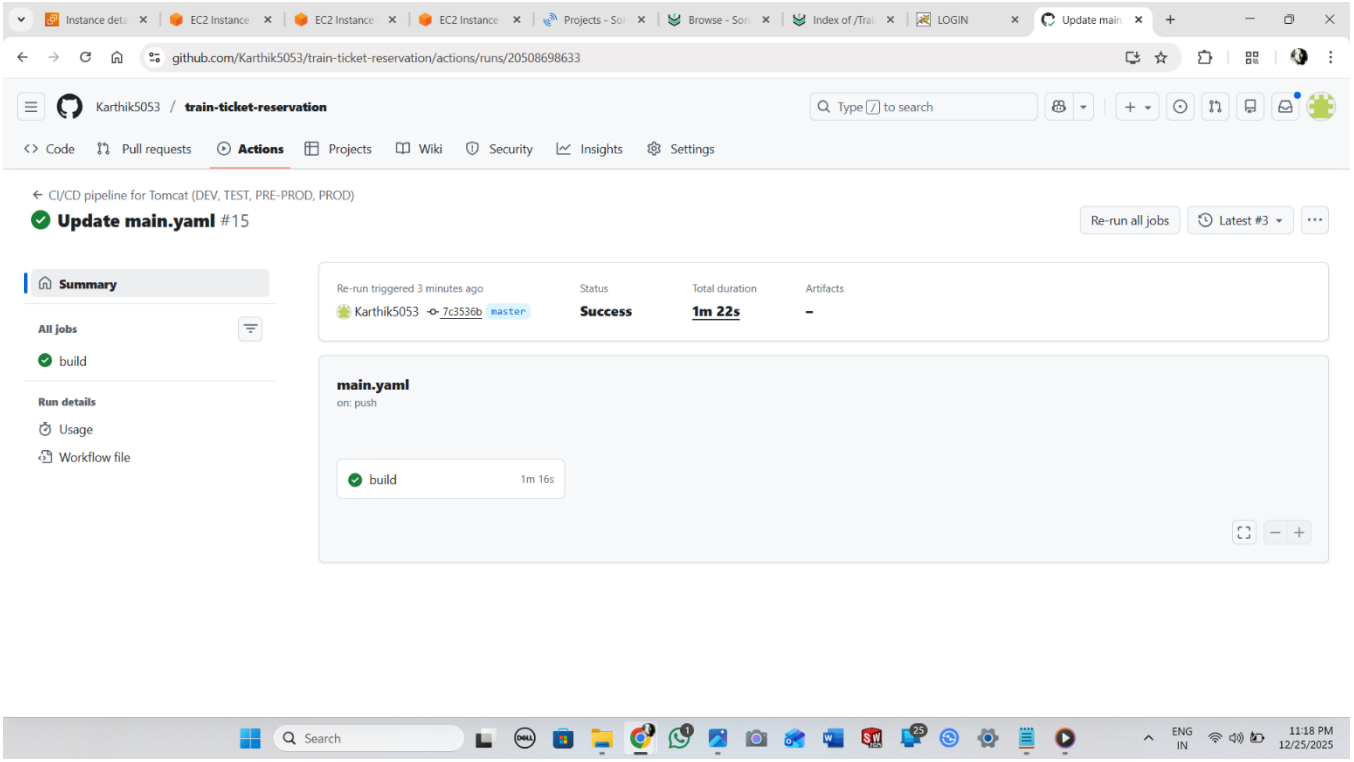


Fig-14: Bulid Github Actions

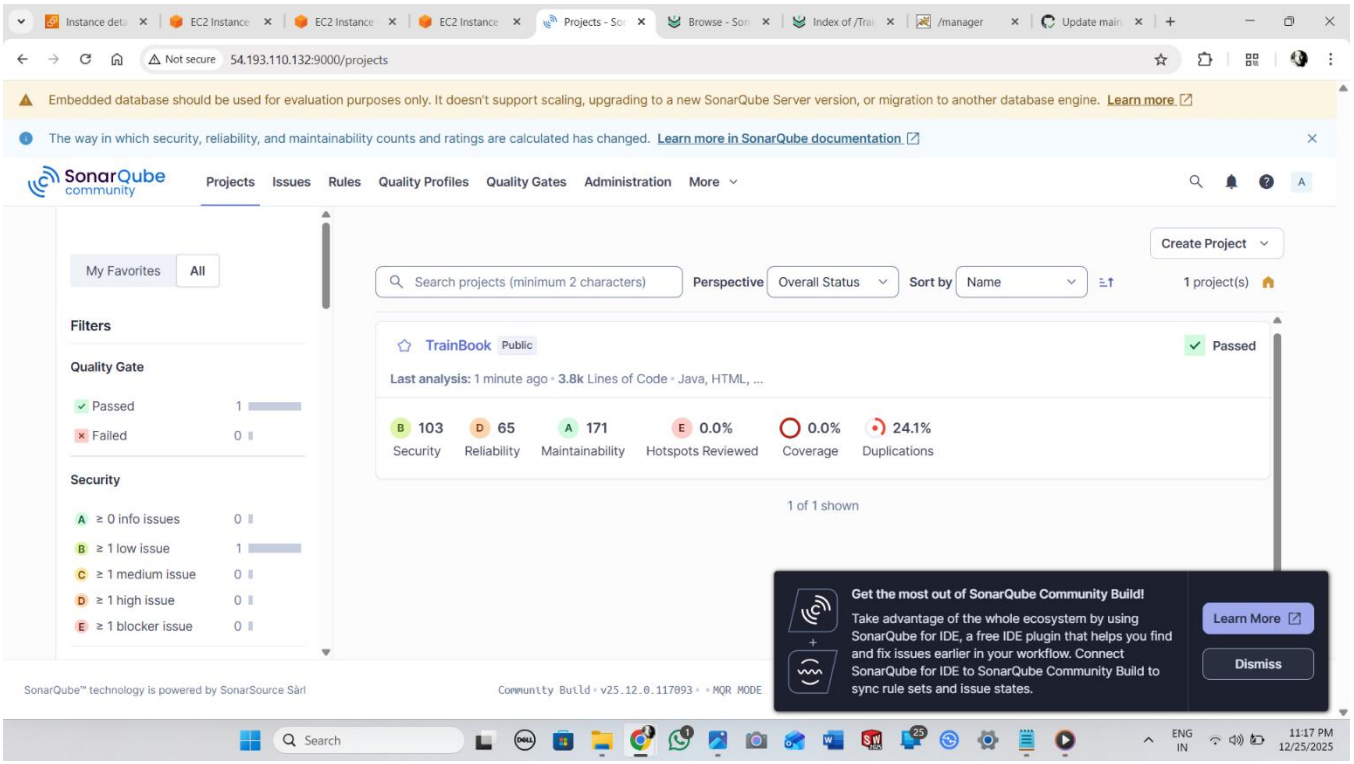


Fig-15: Code Analysis

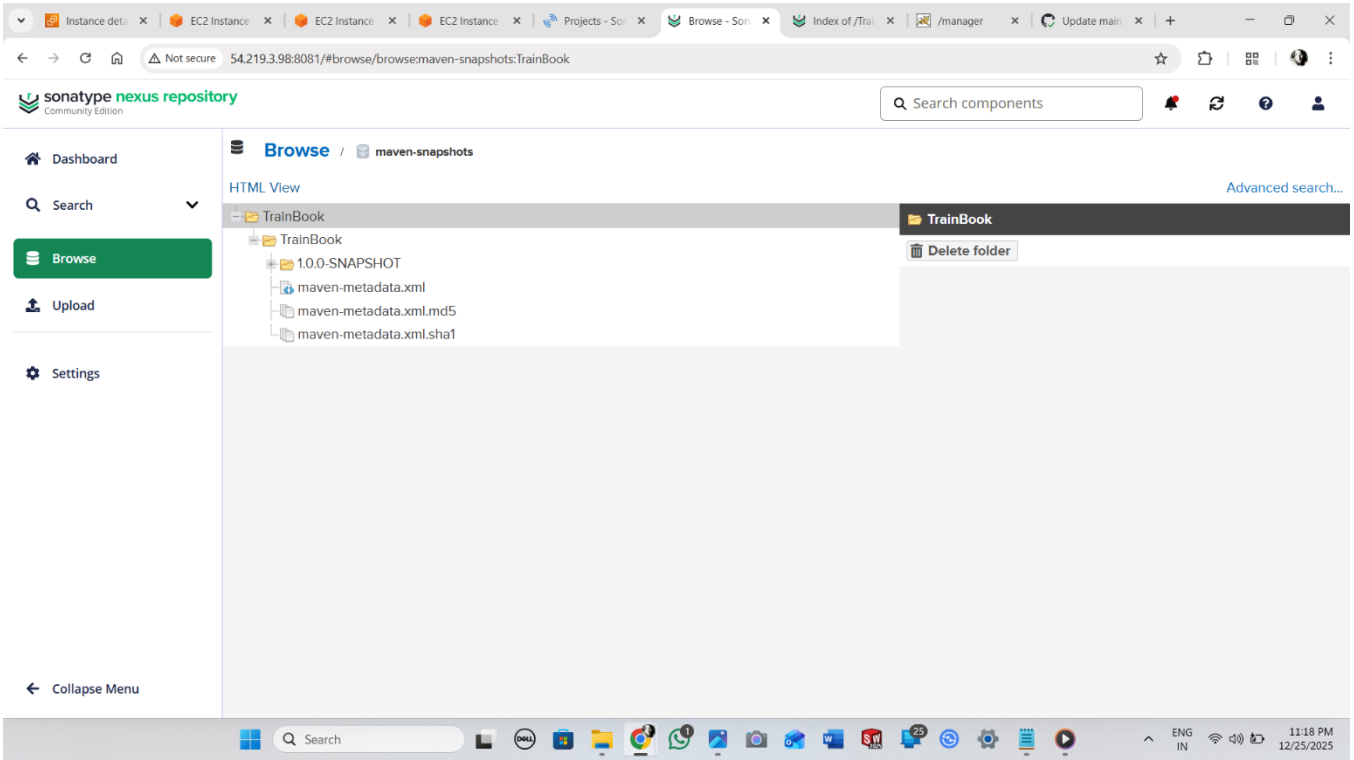


Fig-16: Upload WAR File in Nexus

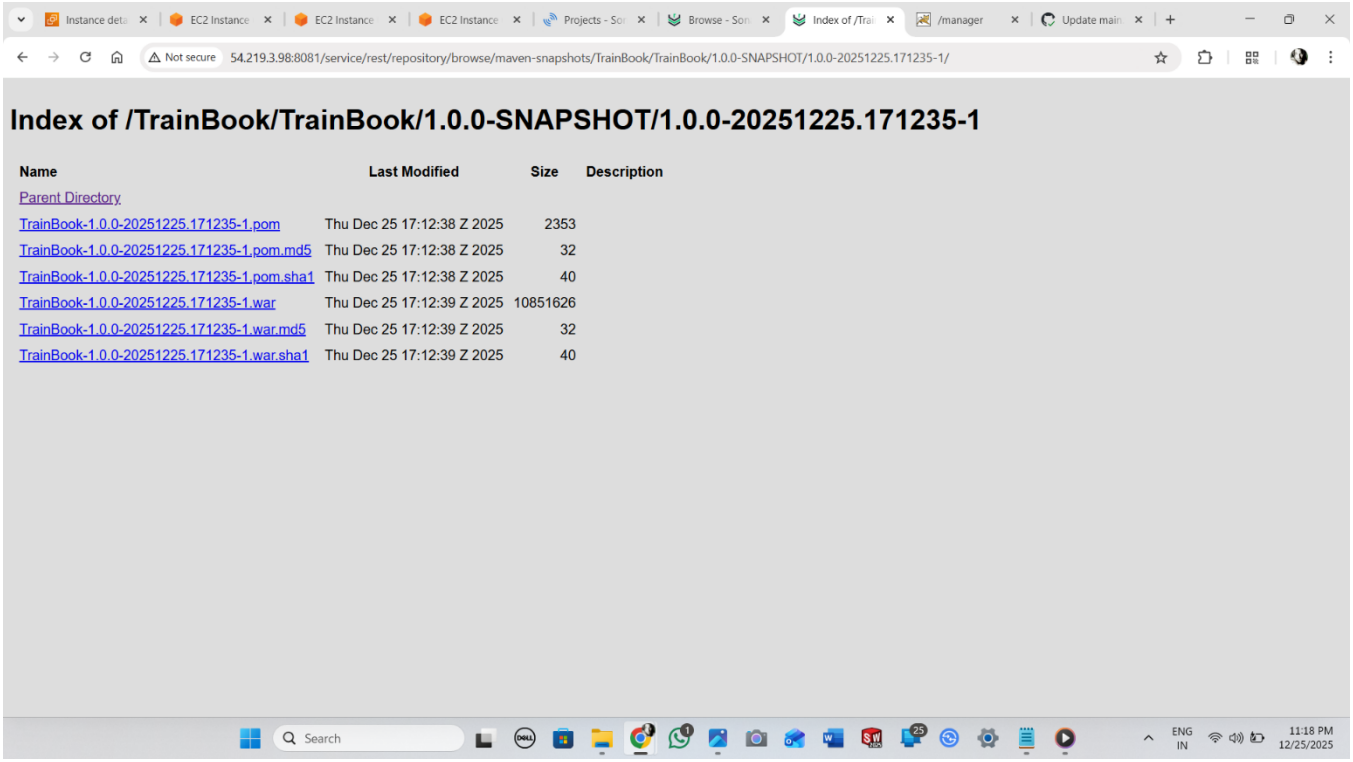


Fig-17: Upload WAR File in Nexus

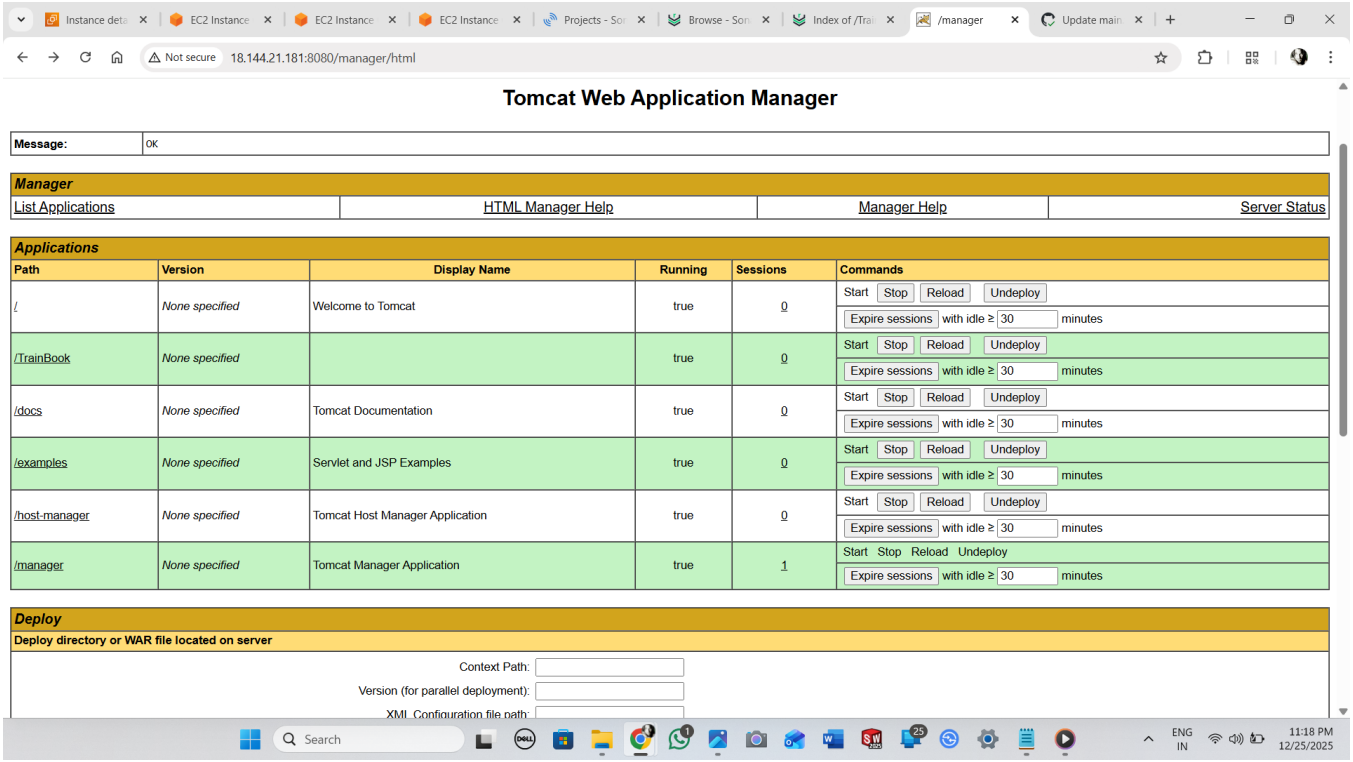


Fig-18: Deploy WAR File in Tomcat

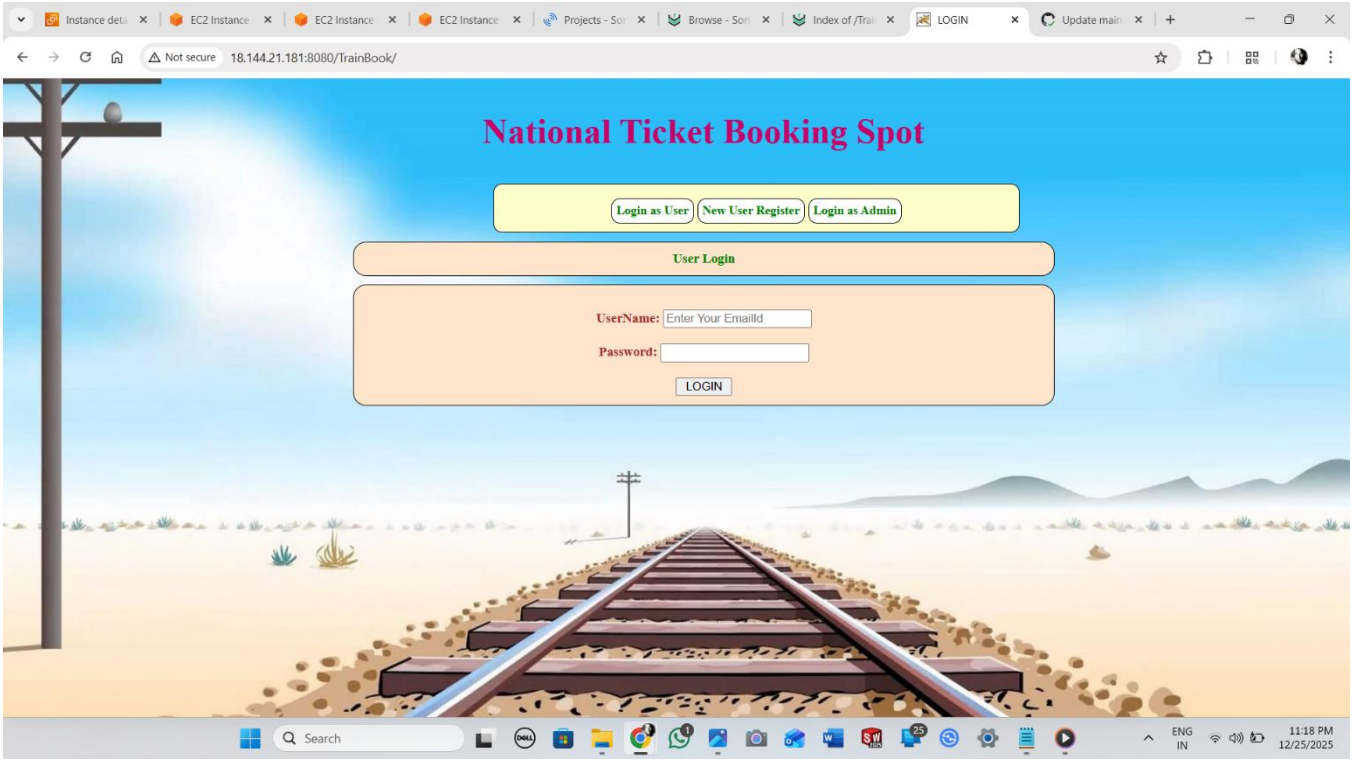


Fig-19: Web Appliaction

CONCLUSION

This project successfully demonstrates the design and implementation of an automated **CI/CD pipeline for a Java web application using GitHub Actions**. The primary goal of the project was to eliminate manual build and deployment efforts by integrating industry-standard DevOps tools into a single automated workflow. The implemented pipeline ensures consistency, reliability, and efficiency throughout the software delivery lifecycle.

By using **Apache Maven**, the project achieves a structured and repeatable build process through well-defined lifecycle phases such as validation, compilation, testing, packaging, and verification. This ensures that the application is built and tested in a controlled environment, reducing the chances of build failures and runtime issues.

The integration of **SonarQube** enhances the overall quality of the application by performing static code analysis during the CI process. It helps identify bugs, code smells, and potential security vulnerabilities at an early stage. This proactive approach improves code maintainability and prevents poor-quality code from progressing further in the pipeline.

The use of **Nexus Repository** provides centralized artifact management, allowing build outputs to be stored, versioned, and reused across environments. This separation between build and deployment aligns with DevOps best practices and ensures that the same verified artifact is used consistently, reducing deployment-related risks.

Automated deployment to **Apache Tomcat** completes the CI/CD process by enabling continuous delivery of the application. Using the Tomcat Manager API, the pipeline deploys or updates the application without manual server intervention, ensuring faster and more reliable releases.

Overall, this project demonstrates a practical and real-world implementation of CI/CD principles. The pipeline improves development speed, enforces code quality standards, minimizes human errors, and enhances deployment reliability. The solution provides a strong foundation that can be extended further with features such as quality gate enforcement, multi-environment deployments, containerization, and advanced security integrations. This project highlights the importance of automation in modern software development and showcases the effective use of DevOps tools to achieve continuous integration and continuous deployment.